

MIDNIGHT FIRESIDE DEV HANG · 15 JULY 2026

NIGHTGATE × NIGHTPASS

How Midnight fits into SAP CAP and how i built a ZK battery passport solution on top of it.

@odatano/nightgate

CAP plugin: indexer + wallet + OData V4

attestation-vault

one Compact contract: attest · disclose · prove

NIGHTPASS

EU battery passport app (Reg. 2023/1542)

Maximilian Weber · ODATANO · github.com/ODATANO



Three layers, one rule: the payload never touches the chain.

APP

NIGHTPASS

Battery domain · cockpit + viewer · tier gating · QR resolver



CAP PLUGIN

@odatano/nightgate (NIGHTGATE)

Crawler → CAP entities · worker-thread wallet · OData V4 · ships the contract



CHAIN

Midnight (preview / preprod)

Substrate node · GraphQL indexer · proof server · Lace — fees in DUST

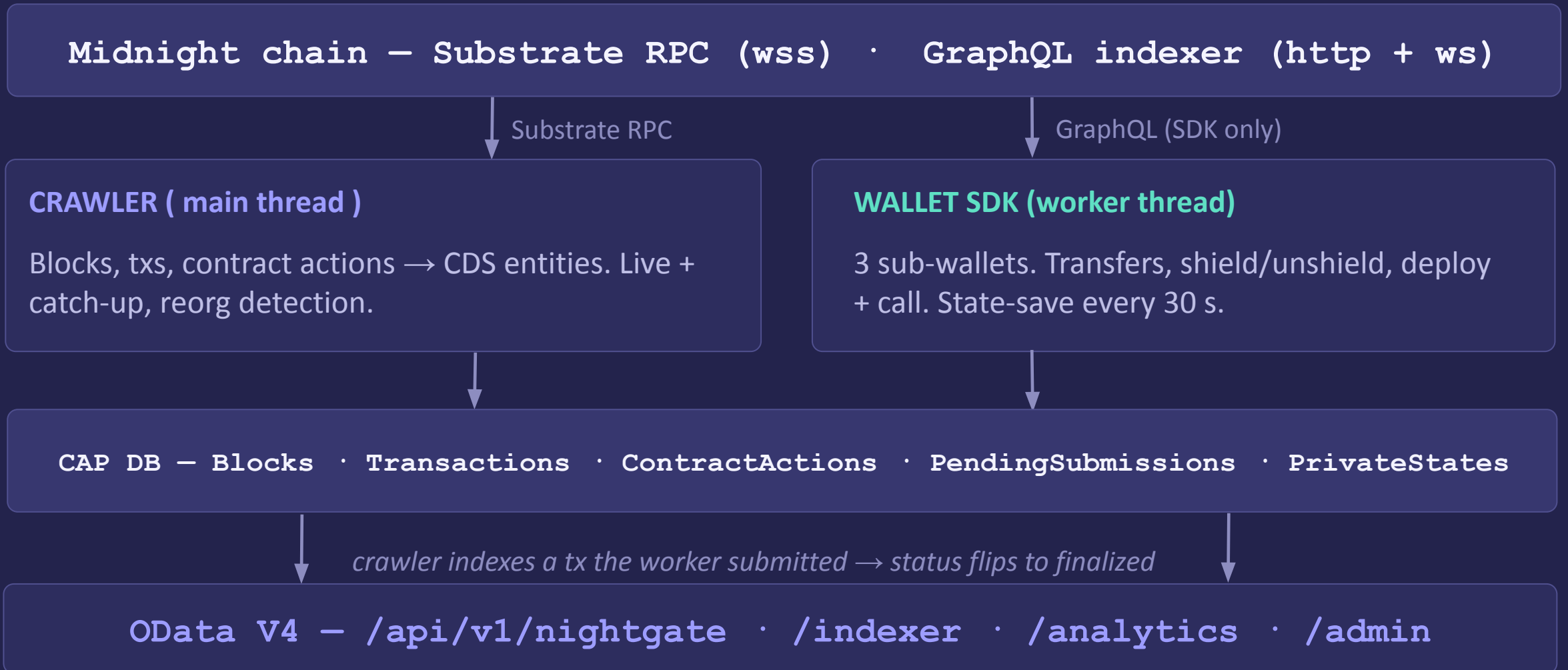
NEVER ON-CHAIN (private)

- the payload
- field values (ZK witnesses)
- supplier identities & footprints
- the sender (shielded wallet)

ON-CHAIN (public)

- payload hash (blake2b-256)
- passportId → hash binding
- Merkle content root
- disclosure grants (level 0/1/2)
- predicate results

Two pipelines, one CAP DB



NIGHTPASS a battery passport that keeps your secrets.

WHY

EU Reg. 2023/1542: every EU battery needs a Digital Product Passport by Feb 2027 — carbon footprint, recycled content, suppliers. All of it competitively sensitive.

WHAT

A Consumer like OData Service & App on top of NIGHTGATE: passports live encrypted in the ERP; Midnight anchors hashes, grants and proofs

ONE PASSPORT'S LIFECYCLE

1 Create

ERP data →
payloadHash, encrypted
off-chain (no tx)



2 Anchor

attest · bindPassport ·
anchorContentRoot (3
txs)



3 Disclose / Prove

grants + ZK predicates
per partner



4 Share

QR → tiered view,
verifiable against the
hash

Passport data stays in the ERP — the chain holds only hashes, grants and proofs.

attestation-vault: one Compact contract, five circuits.

`attest(payloadHash, metaHash)`

lock the hash under the attester identity

`bindPassport(passportId, payloadHash)`

binding for QR / resolve lookups

`anchorContentRoot(payloadHash, root)`

pin the Merkle root over provable fields

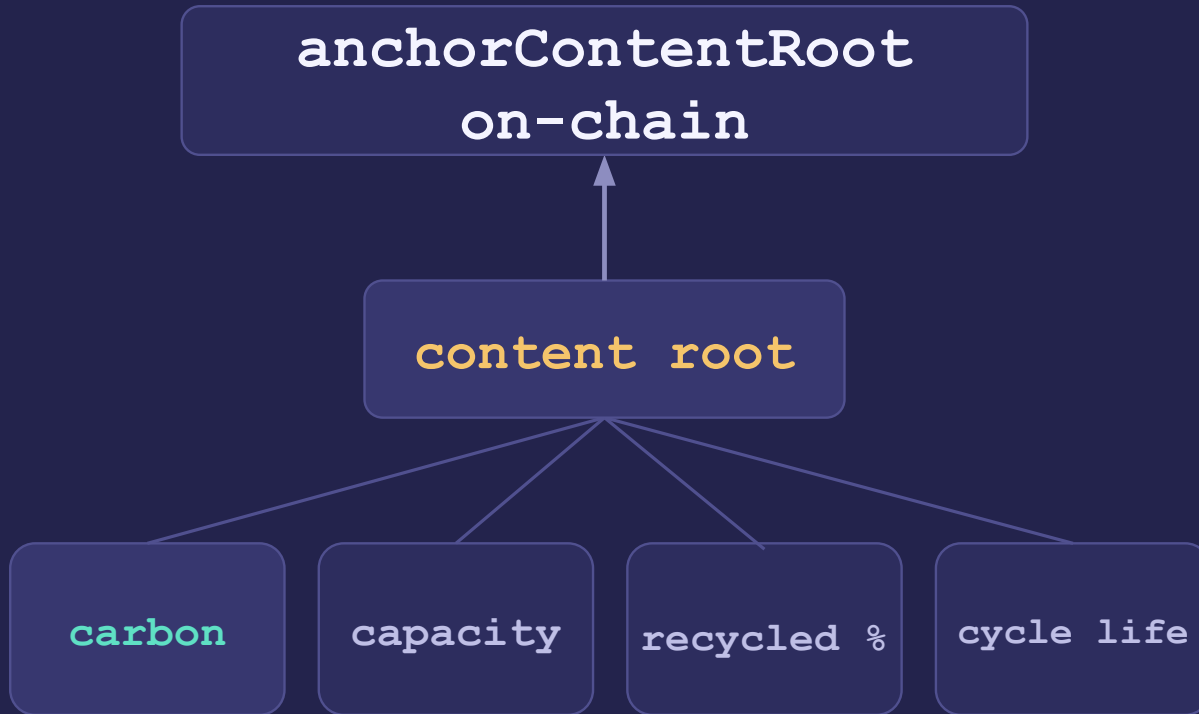
`grantDisclosure / revokeDisclosure`

on-chain ACL: grantee + level 0/1/2

`proveFieldPredicate(hash, key, t, op)`

ZK: field $\leq$ / $\geq$ threshold, bound to the root

Prove “carbon ≤ 40 kg/kWh” without revealing the value.



leaf = `persistentHash(fieldKey, value)` hashed with the contract's own `pureCircuits`.

- 1 witness the value + its Merkle inclusion path
- 2 in-circuit: recompute the leaf, fold to a root
- 3 assert root == anchored content root
- 4 assert value $\leq / \geq$ threshold

The transaction is the proof. A failing predicate is rejected in-circuit so no tx ever lands on chain.

THANKS FOR LISTENING

Questions?

Live on Midnight Preview & Preprod

```
npm install @odatano/nightgate
```

```
"nightgate": { "network": "preprod" }
```

```
github.com/ODATANO/NIGHTGATE  
github.com/ODATANO/NIGHTPASS  
npmjs.com/package/@odatano/nightgate
```